



Portátiles SoloG

Autor:
Unai Garcia Primo

24 de noviembre de 2024

Índice

1- Introducción-----	3
2- Descripción de la web-----	4
3- Tareas realizadas-----	5
4.1- Desarrollo de la aplicación web funcional-----	6
4.2- Crear imágenes de Docker-----	7
4.3- Crear un entorno Docker Compose-----	9
4.4- Despliegue en Kubernetes-----	10
4.5- Uso de Kubernetes no vistos en clase-----	13
5- Uso de asistentes virtuales-----	15
6- Bibliografía-----	16

1- Introducción

Este proyecto consiste en una aplicación web diseñada para la gestión y visualización de datos, implementada utilizando tecnologías modernas de contenedorización y orquestación. Su objetivo principal es proporcionar una plataforma funcional que combina un servidor web con funcionalidades avanzadas de mensajería y almacenamiento, utilizando datos de repositorios públicos como Kaggle.

La aplicación se centra en la integración de tres componentes principales: un servidor web desarrollado en PHP, una base de datos PostgreSQL para el almacenamiento de información estructurada, y RabbitMQ como sistema de mensajería, permitiendo una comunicación eficiente entre los servicios de la aplicación. Además, se incorporan datos obtenidos de fuentes públicas, como Kaggle, que son procesados y almacenados para su posterior consulta y análisis en la plataforma.

La arquitectura está diseñada con un enfoque modular y escalable. Los servicios principales se implementan como contenedores Docker, facilitando la portabilidad y la independencia del entorno. Estos contenedores se despliegan y gestionan mediante Kubernetes, lo que asegura alta disponibilidad y capacidad de adaptación a diferentes cargas de trabajo. Para el servidor web, se utilizan configuraciones específicas de Apache, mientras que la base de datos se inicializa automáticamente mediante scripts SQL predefinidos. RabbitMQ, por su parte, añade un nivel adicional de funcionalidad al actuar como intermediario de mensajes en procesos críticos.

El proyecto también incluye Kubernetes, permitiendo el despliegue, la gestión y la escalabilidad de los servicios que componen la aplicación. Mediante el uso de archivos de configuración YAML, cada componente, como el servidor web, la base de datos PostgreSQL y RabbitMQ, se define como un despliegue independiente. Kubernetes garantiza la alta disponibilidad de los servicios mediante la creación de réplicas y el equilibrio de carga, mientras que los volúmenes persistentes aseguran que los datos almacenados en la base de datos y otros recursos críticos permanezcan accesibles incluso en caso de fallos en los contenedores. Además, el uso de servicios de red e ingress proporciona un enrutamiento eficiente, facilitando el acceso externo a la aplicación y la comunicación interna entre los componentes.

Para saber cómo se ejecuta la aplicación se puede mirar el README del repositorio

[Repositorio Github](#)

2- Descripción de la web

La aplicación web desarrollada en este proyecto es una plataforma integral para la compra de portátiles. Su enfoque principal es brindar un entorno amigable donde los usuarios puedan explorar un catálogo de productos, acceder a detalles técnicos, realizar compras y participar activamente en un foro de preguntas y respuestas.

La web cuenta con un catálogo detallado de ordenadores recogidos de una base de datos de Kaggle. Cada producto se presenta con su fabricante, modelo, especificaciones clave (como tamaño, RAM, tamaño, entre otros) y precio. Una vez seleccionado el producto deseado, los usuarios pueden añadirlo al carrito de compras pulsando en comprar. El sistema también permite revisar el resumen de la compra antes de confirmar el pedido. Para almacenar tanto los usuarios como los portátiles que se venden se ha implementado una base de datos PostgreSQL.

Para aprovechar al máximo las funcionalidades de la plataforma, los usuarios tienen la posibilidad de registrarse creando una cuenta personal. Este proceso, sencillo y rápido, les permite almacenar sus productos en el carrito y participar en el foro, gestionar sus preferencias y participar activamente en el foro.

Uno de los aspectos más destacados de la web es su foro interactivo, un espacio diseñado para fomentar la comunicación entre los usuarios. En este foro, los compradores pueden realizar preguntas sobre los productos, compartir experiencias, buscar recomendaciones o resolver dudas técnicas. Se ha utilizado RabbitMQ para la gestión de colas de mensajes. Además el administrador si ve que la misma pregunta se hace con frecuencia o ve conveniente publicar algún aviso en la página, este administrador puede crear un nuevo tema.

En resumen, esta aplicación web funciona como una tienda en línea de ordenadores, ofreciendo herramientas prácticas y un entorno colaborativo que satisface tanto las necesidades de compra como las de interacción y aprendizaje de sus usuarios.

3- Tareas realizadas

Tareas obligatorias

1. Desarrollo de la aplicación funcional: 
2. Crear imágenes de docker: 
3. Crear un entorno Docker Compose: 

Tareas opcionales

7. Despliegue en Kubernetes: 
8. Uso de Kubernetes más allá de lo visto en clase: 

4.1- Desarrollo de la aplicación web funcional

El desarrollo de la aplicación web se ha enfocado en crear una plataforma robusta y funcional para la compra de ordenadores, utilizando una arquitectura basada en contenedores con Docker para asegurar escalabilidad y modularidad. Se ha utilizado Visual Studio Code para programar. En el backend, se implementó un servidor web utilizando PHP con Apache para gestionar las solicitudes de los usuarios, mientras que PostgreSQL se empleó como sistema de gestión de bases de datos, garantizando la persistencia de los datos del catálogo, usuarios y carritos de compra.

En el frontend, se utilizó HTML, CSS y JavaScript para crear una interfaz limpia, atractiva y fácil de usar. El diseño se centró en la experiencia del usuario, asegurando que la navegación fuera intuitiva, desde la exploración del catálogo hasta la gestión del carrito y la interacción con el foro. Además, el foro de preguntas y respuestas fue optimizado utilizando RabbitMQ como sistema de mensajería asíncrona, garantizando una gestión eficiente de las publicaciones y respuestas incluso en momentos de alta actividad, lo que mejora la interactividad y la experiencia del usuario. Todo el sistema está contenido en contenedores Docker, con un entorno Docker Compose que facilita el despliegue, y Kubernetes se configura como una opción para escalar la aplicación según sea necesario.

4.2- Crear imágenes de Docker

En la parte de la creación de imágenes Docker, se configuraron varios contenedores para los distintos componentes de la aplicación, garantizando una arquitectura modular y escalable. Se crearon imágenes para el servidor web, la base de datos y un servicio adicional para la mensajería del foro utilizando RabbitMQ. Cada imagen fue diseñada para ejecutar de manera aislada y específica una parte del sistema, asegurando que cada contenedor tuviera un propósito claro y no interfiriera con el funcionamiento de otros componentes.

La imagen del servidor web utiliza la versión `php:7.4-apache`, e instala las dependencias necesarias como las bibliotecas para PostgreSQL (`libpq-dev`, `pgsql`, `pdo_pgsql`) y RabbitMQ (`librabbitmq-dev`), además de herramientas como `git` y `unzip`. Se instala Composer para gestionar las dependencias de PHP y se configuran permisos adecuados para el directorio de la aplicación. Además, se copia un archivo de configuración de Apache personalizado y los archivos de la aplicación al contenedor. Finalmente, se ejecuta `composer install` para instalar las dependencias y se habilita el módulo `rewrite` de Apache, necesario para las reescrituras de URL. [Unaigarcia02/web](#)

Para la base de datos se configura una imagen de Docker basada en la imagen oficial de PostgreSQL. Primero, copia un script SQL (`juegos.sql`) al directorio de inicialización de PostgreSQL (`/docker-entrypoint-initdb.d/`), lo que permite que el script se ejecute automáticamente al iniciar el contenedor y configure la base de datos. Luego, establece las variables de entorno necesarias para la conexión a la base de datos, incluyendo el nombre de usuario (`POSTGRES_USER`), la contraseña (`POSTGRES_PASSWORD`) y el nombre de la base de datos (`POSTGRES_DB`). Finalmente, expone el puerto 5432, que es el puerto predeterminado para PostgreSQL, permitiendo la conexión al contenedor desde el exterior. [Unaigarcia02/postgres](#)

A la hora de hacer el foro de mensajes se utiliza el `dockerfile-rabbit`, este Dockerfile configura una imagen basada en la imagen oficial de RabbitMQ con la interfaz de gestión habilitada (`rabbitmq:management`). Primero, habilita el plugin de gestión web de RabbitMQ mediante el comando `rabbitmq-plugins enable --offline rabbitmq_management`. Luego, expone los puertos necesarios: el puerto 5672 para la comunicación AMQP y el puerto 15672 para acceder a la interfaz web de gestión. Además, establece las variables de entorno `RABBITMQ_DEFAULT_USER` y `RABBITMQ_DEFAULT_PASS` para configurar las credenciales predeterminadas del usuario (`guest`). Finalmente, define el comando por defecto para iniciar el servidor RabbitMQ con `rabbitmq-server`, lo que arranca el servicio cuando el contenedor se ejecuta. [unaigarcia02/rabbit](#)

Por último se utiliza una imagen de docker que utiliza Python 3.10 para descargar los datos de kaggle. Primero, establece el directorio de trabajo como `/app`. Luego, actualiza los paquetes del sistema e instala `python3-distutils` para asegurarse de que las herramientas necesarias para instalar dependencias estén presentes. A continuación, instala las librerías de Python necesarias, incluyendo `kaggle` y `pandas`, para permitir la interacción con la API de Kaggle y el procesamiento de datos. Crea un directorio `.kaggle` en el directorio raíz para almacenar las credenciales de Kaggle. Luego, copia el script de descarga y procesamiento de datos (`kagglescript.py`) al contenedor en el directorio de trabajo. Este archivo de python descarga una base de datos de [Kaggle](#) y la convierte a json para que el php pueda leerlo. Finalmente, define el comando por defecto para ejecutar el script de Kaggle y procesar los datos, seguido de `tail -f /dev/null` para mantener el contenedor en ejecución después de la ejecución del script. [unaigarcia02/kaggle-data](#)

4.3- Crear un entorno Docker Compose

Para levantar los múltiples contenedores se ha creado un `docker-compose.yml` en el que se han declarado los servicios a levantar. Este archivo define cuatro servicios esenciales para la aplicación: **db** (PostgreSQL), **web** (Aplicación Web), **kaggle-data** (Procesamiento de Kaggle) y **rabbitmq** (Mensajería con RabbitMQ). Cada servicio está cuidadosamente configurado para garantizar la correcta interacción entre ellos. El servicio **db** usa una imagen de PostgreSQL y un Dockerfile personalizado (`dockerfile-sql`) para configurar la base de datos, establecer credenciales de acceso, y montar volúmenes persistentes para los datos. Además, expone el puerto 5432 para permitir conexiones a la base de datos y está conectado a la red `mynetwork` junto con el resto de los servicios.

El servicio **web** contiene la aplicación web, que se construye desde un Dockerfile (`dockerfile-web`) personalizado, y depende de RabbitMQ para la mensajería. Expone el puerto 8080 en el contenedor y se conecta a la misma red, permitiendo que interactúe con otros servicios como la base de datos y el contenedor de Kaggle. Además, monta el volumen `data` para almacenar los datasets. El servicio **kaggle-data** está dedicado a ejecutar scripts que procesan datos desde Kaggle. Se construye utilizando un Dockerfile (`dockerfile-kaggle`) específico y también monta el volumen `data` para acceder a los datasets que procesa.

Finalmente, el servicio **rabbitmq** construido por otro Dockerfile (`dockerfile-rabbit`) usa la imagen oficial de RabbitMQ con el plugin de gestión habilitado, lo que permite la gestión del sistema a través de una interfaz web. Exponiendo los puertos necesarios para la comunicación AMQP y la interfaz de gestión web, este servicio está configurado para reiniciarse automáticamente en caso de fallos y está vinculado a la red `mynetwork` para facilitar la comunicación con los otros contenedores. Todos los servicios comparten la red `mynetwork` y utilizan el archivo `.secrets.txt` para gestionar configuraciones sensibles de manera más segura.

Para desplegar el proyecto nos tenemos que situar en la raíz del proyecto, donde están todos los dockerfiles, y ejecutamos “***docker-compose up -d --build***”.

4.4- Despliegue en Kubernetes

Para el despliegue en Kubernetes se ha utilizado minikube. Es una herramienta que nos permite crear un cluster de manera local, además de ser fácil de usar e intuitiva. El único inconveniente es que no viene incluido con Ingress, lo cual hay que instalar ejecutando “***minikube addons enable ingress***”.

El despliegue de la aplicación en Kubernetes implica la creación de varios recursos de Kubernetes que permiten gestionar, escalar y asegurar la ejecución de los servicios de la aplicación. Para empezar, se definen **Pods**, que son las unidades más pequeñas en Kubernetes y agrupan contenedores relacionados.

Para desplegarlo se ha utilizado **release.yaml**, el cual tiene dentro todos los Kubernetes necesarios unidos. Este archivo define varios recursos necesarios para desplegar una aplicación completa que incluye una base de datos PostgreSQL, un servicio de mensajería RabbitMQ, un servicio de procesamiento de datos de Kaggle y una aplicación web. Los recursos clave incluyen **Services**, **Deployments**, **Persistent Volumes (PV)**, **Persistent Volume Claims (PVC)**, **ConfigMap**, y **Ingress**.

Servicios (Services)

El archivo comienza con la definición de tres **Services** para exponer los Pods correspondientes dentro del clúster de Kubernetes:

1. **db**: Expone el puerto 5432 de PostgreSQL para que otros Pods puedan acceder a la base de datos.
2. **rabbitmq**: Expone los puertos 5672 (AMQP) y 15672 (gestión web) para que los clientes y otros servicios puedan interactuar con RabbitMQ.
3. **web**: Expone el puerto 8080 para la aplicación web, permitiendo que los usuarios accedan a la interfaz de la aplicación.

Cada uno de estos servicios tiene un **selector** que hace coincidir los Pods correspondientes por las etiquetas **service: db**, **service: rabbitmq** y **service: web**, respectivamente.

Despliegues (Deployments)

El archivo define cuatro **Deployments**, que especifican cómo deben ejecutarse los Pods correspondientes:

1. **db**: Despliega un Pod que ejecuta PostgreSQL, configurado con las credenciales de acceso definidas en un **ConfigMap**. El Pod tiene un volumen montado para almacenar los datos persistentes de la base de datos.
2. **kaggle-data**: Despliega un Pod que ejecuta el servicio de procesamiento de datos de Kaggle, que tiene acceso a las mismas credenciales almacenadas en el **ConfigMap** y a un volumen persistente para los datos.
3. **rabbitmq**: Despliega un Pod que ejecuta RabbitMQ, también con acceso a un volumen persistente para los datos de RabbitMQ.
4. **web**: Despliega un Pod que ejecuta la aplicación web, también configurado con las credenciales necesarias desde el **ConfigMap** y con acceso a un volumen persistente para los datos.

Cada Deployment especifica un único contenedor para los Pods, configurado con las variables de entorno necesarias, incluyendo las credenciales y claves para acceder a los servicios de PostgreSQL y Kaggle.

ConfigMap

El **ConfigMap** llamado **secrets-txt** almacena las variables de entorno sensibles, como las claves y credenciales necesarias para acceder a la base de datos PostgreSQL y la API de Kaggle. Estas variables son consumidas por los contenedores en los Pods a través de referencias a los valores almacenados en el ConfigMap.

Volúmenes Persistentes (Persistent Volumes)

El archivo también define tres **Persistent Volumes** (PV), cada uno de 5 GB, para almacenar los datos de forma persistente:

1. **data-pv**: Utilizado por el servicio de Kaggle y la aplicación web para almacenar datos compartidos.
2. **pg-data-pv**: Utilizado por el servicio de PostgreSQL para almacenar datos de la base de datos.
3. **rabbitmq-data-pv**: Utilizado por RabbitMQ para almacenar sus datos de mensajería.

Cada volumen tiene una política de recuperación de datos **Retain**, lo que significa que no se eliminarán cuando se eliminen los Pods que los usan. Estos volúmenes están configurados con un **hostPath**, lo que significa que Kubernetes utilizará el sistema de archivos local de los nodos del clúster para almacenar los datos.

Persistent Volume Claims (PVC)

Se crean tres **Persistent Volume Claims** (PVC), uno para cada volumen persistente, lo que permite a los Pods solicitar almacenamiento específico. Cada

PVC está configurado con un tamaño de 5 GB y solicita un volumen de almacenamiento con la clase **standard**.

Ingress

Finalmente, el archivo define un **Ingress** llamado **app-ingress**, que gestiona el acceso HTTP a los servicios de la aplicación:

- La ruta **/** se redirige al servicio **web** en el puerto 80.
- La ruta **/rabbitmq** se redirige al servicio **rabbitmq** en el puerto 15672 (interfaz de gestión de RabbitMQ).

El uso de un Ingress permite gestionar el tráfico HTTP de manera más flexible y avanzada, como la posibilidad de aplicar reglas de enrutamiento y configuraciones de SSL.

En resumen, este archivo de Kubernetes define una infraestructura completa para desplegar y gestionar una aplicación web con servicios de base de datos, mensajería y procesamiento de datos, asegurando que todos los componentes estén correctamente configurados y puedan escalar según sea necesario.

Para desplegar los Kubernetes nos tenemos que situar sobre la carpeta "Kubernetes", iniciar minikube y ejecutar **"kubectl apply -f release.yaml"**.

4.5- Uso de Kubernetes no vistos en clase

Algo que no hemos visto en clase es el uso del configmap, que se utiliza para gestionar variables de entorno. En [release.yaml](#), se utiliza un **ConfigMap** llamado **secrets-txt** para gestionar información como las credenciales de acceso a **Kaggle** y **PostgreSQL**.

```
apiVersion: v1
data:
  KAGGLE_KEY: 0045be890d66bee7cf19e46a282f0cdc
  KAGGLE_USERNAME: unaiprimo
  POSTGRES_PASSWORD: admin
  POSTGRES_USER: juegosacceso
kind: ConfigMap
metadata:
  labels:
    service: db-secrets-txt
  name: secrets-txt
```

Explicación:

- **ConfigMap secrets-txt:** Este **ConfigMap** contiene claves de configuración que incluyen:
 - **KAGGLE_KEY:** La clave de API para Kaggle.
 - **KAGGLE_USERNAME:** El nombre de usuario de Kaggle.
 - **POSTGRES_PASSWORD:** La contraseña para la base de datos PostgreSQL.
 - **POSTGRES_USER:** El nombre de usuario para la base de datos PostgreSQL.

Estas configuraciones se pasan a los contenedores como variables de entorno mediante la referencia a este **ConfigMap** en los contenedores de los servicios.

Uso del ConfigMap en los contenedores:

Dentro de los despliegues (Deployments) de los servicios como **db**, **kaggle-data**, y **web**, se hace referencia al **ConfigMap secrets-txt** para establecer las variables de entorno:

```
env:
  - name: KAGGLE_KEY
    valueFrom:
      configMapKeyRef:
```

```
    key: KAGGLE_KEY
    name: secrets-txt
- name: KAGGLE_USERNAME
  valueFrom:
    configMapKeyRef:
      key: KAGGLE_USERNAME
      name: secrets-txt
- name: POSTGRES_PASSWORD
  valueFrom:
    configMapKeyRef:
      key: POSTGRES_PASSWORD
      name: secrets-txt
- name: POSTGRES_USER
  valueFrom:
    configMapKeyRef:
      key: POSTGRES_USER
      name: secrets-txt
```

¿Qué hace esto?

- Cada contenedor obtiene su respectiva variable de entorno (**KAGGLE_KEY**, **KAGGLE_USERNAME**, **POSTGRES_PASSWORD**, **POSTGRES_USER**) a partir del **ConfigMap** llamado **secrets-txt**.
- Esta forma de gestionar las configuraciones permite que los valores sean fácilmente modificables en el **ConfigMap** sin necesidad de reconstruir o redespargar los contenedores.

5- Uso de asistentes virtuales

Para hacer este proyecto se ha utilizado bastante GPT-4, me ha facilitado mucho la programación de php, html y css. Cuando he tenido una pregunta también le he consultado, pero el uso de las imágenes de docker y la gestión de la base de datos ha sido cosa mía y de la documentación de PostgreSQL.

6- Bibliografía

GPT-4. <https://chatgpt.com/>

PostgreSQL. <https://www.postgresql.org/docs/>